

RBAC Design Document for Hybrid VectorDB + PostgreSQL Architecture

1. Objective

This document defines the Role-Based Access Control (RBAC) model for the enterprise AI assistant platform using:

- **Qdrant Vector Database** for unstructured document retrieval
- **PostgreSQL** for structured operational and metadata storage
- Hybrid retrieval architecture supporting SOPs, tickets, policies, and enterprise knowledge

The RBAC system ensures:

- Secure access to enterprise documents and tickets
- Department-level and role-level isolation
- Fine-grained filtering during retrieval
- Compliance with enterprise governance and audit requirements

2. RBAC Architecture

2.1 Core Components

Component	Purpose
Authentication Layer	User login via SSO/OAuth/SAML
Authorization Layer	Role and permission evaluation
PostgreSQL	Stores users, roles, permissions, mappings
Qdrant VectorDB	Stores embeddings + metadata filters
Retrieval Layer	Enforces access filters during search
Audit Layer	Tracks access requests and violations

3. RBAC Flow

Step 1 — User Authentication

Users authenticate using:

- SSO
- Azure AD
- Okta
- LDAP
- OAuth2

The identity provider returns:

- User ID
- Roles
- Department
- Groups
- Clearance level

Step 2 — Authorization Resolution

The authorization service retrieves:

- Assigned roles
- Effective permissions
- Department restrictions
- Resource scopes

from PostgreSQL.

Step 3 — Query Enforcement

Before retrieval:

PostgreSQL Path

SQL queries are filtered using:

- Tenant ID
- Department ID
- Project scope
- Role permissions
- Record ownership

Example:

```
SELECT * FROM tickets
WHERE department_id = :dept_id
AND sensitivity_level <= :clearance;
```

VectorDB Path (Qdrant)

Metadata filters are applied during vector search.

Example metadata:

```
{
  "department": "Finance",
  "classification": "Internal",
  "allowed_roles": ["Finance_Admin", "Manager"],
  "tenant_id": "org_001"
}
```

Example Qdrant filter:

```
{
  "must": [
    {
```

```

    "key": "department",
    "match": { "value": "Finance" }
  },
  {
    "key": "allowed_roles",
    "match": { "any": ["Manager"] }
  }
]
}

```

This ensures unauthorized chunks are never retrieved.

4. PostgreSQL RBAC Schema

4.1 Recommended Tables

Table	Purpose
users	Stores user profile and identity
roles	Role definitions
permissions	Permission catalog
user_roles	User-role mapping
role_permissions	Role-permission mapping
departments	Department hierarchy
resource_policies	Access policies
audit_logs	Security and access tracking

4.2 Role Hierarchy Example

Role	Access
Super Admin	Full platform access
Department Admin	Department-level management
Manager	Team-specific access

Employee	Standard document access
Auditor	Read-only compliance access

5. VectorDB RBAC Design

5.1 Required Metadata in Qdrant

Each document chunk stored in Qdrant should contain:

Metadata Field	Purpose
document_id	Parent document reference
tenant_id	Multi-tenant isolation
department	Department restriction
allowed_roles	Role-based filtering
sensitivity_level	Clearance enforcement
source_system	SharePoint/Jira/etc
owner_id	Ownership checks
expiration_date	Policy lifecycle

5.2 Retrieval Enforcement

The retrieval layer must:

1. Resolve user permissions
2. Build metadata filters
3. Apply filters during vector search
4. Reject unauthorized chunks
5. Log access attempts

6. Required ER Diagram Changes

6.1 New RBAC Tables Required

The current ER diagram should be extended with the following tables:

users

user_id (PK)
email
name
department_id
clearance_level
status
created_at

roles

role_id (PK)
role_name
description
created_at

permissions

permission_id (PK)
permission_name
resource_type
action

user_roles

user_role_id (PK)
user_id (FK)
role_id (FK)

assigned_at

role_permissions

role_permission_id (PK)

role_id (FK)

permission_id (FK)

resource_policies

policy_id (PK)

resource_type

resource_id

allowed_roles

classification

created_at

audit_logs

audit_id (PK)

user_id

resource_id

action

status

timestamp

6.2 Recommended Relationships

Relationship	Description
users → departments	Many-to-one
users ↔ roles	Many-to-many
roles ↔ permissions	Many-to-many
resources ↔ policies	One-to-many

users → audit_logs One-to-many

7. Security Recommendations

Mandatory Controls

- Enforce RBAC before retrieval
- Never expose raw vector search without metadata filtering
- Use row-level security (RLS) in PostgreSQL
- Encrypt sensitive metadata
- Maintain immutable audit logs
- Support policy-based escalation
- Cache permissions carefully with expiration

8. Recommended Enhancements

Enhancement	Benefit
Attribute-Based Access Control (ABAC)	Dynamic policy enforcement
Document Classification Engine	Automated sensitivity tagging
Time-based Access	Temporary access windows
Just-in-Time Privileges	Reduce standing permissions
Policy Versioning	Compliance tracking

9. Conclusion

The proposed RBAC architecture enables secure enterprise-grade retrieval across both structured and unstructured systems.

PostgreSQL acts as the authoritative authorization store, while Qdrant enforces retrieval-time filtering using metadata-aware vector search.

This design ensures:

- Secure document isolation
- Department-aware retrieval
- Compliance-ready auditing
- Scalable multi-tenant support
- Safe enterprise AI deployment